

Soft querying powered by user-defined functions in *J-CO-QL*⁺

Paolo Fosci, Giuseppe Psaila^{*}

University of Bergamo, Dept. of Management, Information and Production Engineering, Viale Marconi 5, Dalmine 24044, BG, Italy



ARTICLE INFO

Article history:

Received 5 May 2022

Revised 17 February 2023

Accepted 7 May 2023

Available online 12 May 2023

Keywords:

Soft querying on JSON data sets

User-defined functions

Fuzzy operators powered by user-defined functions

ABSTRACT

Soft querying on databases (i.e., selecting data items that partially match selection conditions) was investigated on top of classical relational databases in past research works; however, constraints and limitations posed by relational DBMSs significantly limited the practical effects of this research. The advent of JSON as the format for representing and sharing data over the Internet, together with the birth of JSON document stores (a specific category of NoSQL databases), is now changing the panorama. In fact, the need to integrate and query large JSON data sets is now calling for novel and powerful tools for managing and integrating JSON data sets in a flexible way. At the University of Bergamo (Italy), we are devising the *J-CO* Framework, which is a platform-independent tool that relies on a high-level and general-purpose language named *J-CO-QL*⁺: among all its features, it provides capabilities towards “soft querying” of JSON documents.

However, a general-purpose language, although extremely powerful, cannot provide support for domain-specific computations that often relies on procedural algorithms. In this paper, we show how supporting user-defined functions actually empowers *J-CO-QL*⁺ users towards applying soft querying on JSON data sets. User-defined functions written both in JavaScript and in Java are accepted by the *J-CO-QL*⁺ Engine: in this paper, we present how to define them and the different execution performance.

© 2023 Elsevier B.V. All rights reserved.

1. Introduction

JSON (acronym for JavaScript Object Notation, see [14]) has become the most used format to represent data sets in various application contexts: indeed, it is easy to find JSON data sets published on web sources and Open Data portals, as well as they are provided by APIs (Application Programming Interfaces) of web services. It is possible to argue the reasons of such a success: (i) the syntax is simple and easy to parse; (ii) a JSON document is a serialized representation of an object managed by an object-oriented programming language, thus JSON data sets can be easily processed by programming languages; (iii) JSON data sets are easier to process than XML documents. The diffusion of JSON has brought to develop a specific category of NoSQL databases named *document databases* (a “document” is a JSON object). Various DBMSs (Database Management Systems) able to store and query JSON documents are now available; the most famous is *MongoDB*¹, but *CouchDB*² (see [2]) is a valuable tool too (chosen as the DBMS for

the *HyperLedger Fabric* Blockchain platform, see [15]). A DBMS able to natively deal with JSON documents is named “JSON store” as well.

This incredible variety of JSON data sets to integrate and query asks for powerful tools oriented to analysts. In fact, we identified the following scenario. (i) Each single JSON store provides its own query language, so the integration of data sets stored in different JSON stores is not immediate. (ii) If JSON data sets are captured directly from the Internet, it could be not obvious being able to store them into a JSON store for preprocessing (for example, to avoid to make the store dirty). (iii) Analysts would like to have a high-level query language, while JSON stores usually provide low-level query languages.

At the University of Bergamo (Italy), in 2017 we started a research project whose result is the *J-CO* Framework (see [40]): it is a platform-independent suite of software tools to gather, store, manage and query JSON data sets. The core of the framework was (and is) its query language, originally named *J-CO-QL*: the original design goal (see [5,4]) was coping with geo-tagging and heterogeneity of documents within the same data set. Then, we considered that it was necessary to introduce constructs for “soft querying” JSON documents, so as to allow analysts to perform possibly-qualitative queries based on partial matching (see

^{*} Corresponding author.

¹ *MongoDB* web site, accessed on 16/02/2023: <https://www.mongodb.com/>.

² *CouchDB* web site, accessed on 16/02/2023: <https://couchdb.apache.org/>.

[41,19,20]), by evaluating membership degrees of *JSON* documents to fuzzy sets.

While introducing soft-querying capabilities, we realized that some syntactic and semantic choices were either ambiguous or not immediately comprehensible. Thus, in 2022 (see [21,22]), we started the development of *J-CO-QL*⁺: it is based on the same approach followed for the original *J-CO-QL*, but improves syntax and semantics of key constructs, as well as it better supports fuzzy-set evaluation on *JSON* documents and allows for defining user-defined functions. This latter feature, in particular, is very useful to extend the basic capabilities of the language: indeed, *J-CO-QL*⁺ is a powerful high-level language for manipulating collections of *JSON* documents; however, domain-specific functionalities cannot be provided by a general purpose language as built-in functions; so, *J-CO-QL*⁺ allows analysts to create and use functions, so as to perform complex soft queries.

The first contribution of the paper is to present the current interpretation of the concept of user-defined function within *J-CO-QL*⁺: this concept has reached a higher level of maturity, in comparison with the early idea presented in [19] and is fully implemented. In particular, now both JavaScript and Java are supported as programming languages to write user-defined functions; this way, analysts are provided with a very high degree of flexibility, because JavaScript functions are easier to write but slower to execute (compared with Java functions), while Java functions are more complex to write (compared with JavaScript functions) but faster to execute.

The second contribution of the paper is to show how soft querying can become effective for processing practical data sets. We consider two different dimensions of effectiveness: (i) user-defined functions allow analysts to introduce (possibly complex) domain-specific procedural computations, to be jointly exploited both for classical crisp querying and for soft querying; (ii) Java user-defined functions are very quick to execute, so as they can be actually exploited for soft querying large data sets quickly with *J-CO-QL*⁺.

The remainder of the paper is organized as follows. Section 2 introduces the related work. Section 3 provides the background of the paper: it gives a brief introduction to basic concepts concerning fuzzy sets (in Section 3.1), as well as it introduces the *J-CO* Framework and its data and execution models for the *J-CO-QL*⁺ language (in Section 3.2). Section 4 shows how *J-CO-QL*⁺ supports user-defined functions. Section 5 presents a case study and the *J-CO-QL*⁺ script for soft querying a data set. Section 6 discusses how user-defined functions are implemented and study performances. Finally, Section 7 draws conclusions and future work.

2. Related work

Our research work originates from the previous research works on soft querying on relational databases. Under the hypothesis that a relational database is already available, to perform soft querying it is necessary to develop query languages that work on top of it; typically, these languages extend the concept of “selection condition”, as argued by [3], so as to formulate “soft queries” in which selection conditions can rely on vague predicates. This way, queries become tolerant to thresholds; for example, given a non-soft (crisp) predicate `age <=25` to select young people, a 26-year old person would not be selected, while this person could be of interest too. To express vague concepts and relationships, fuzzy sets (see [47]) provide a suitable formal framework. Shortly, an item belongs to a fuzzy set with a “membership degree” in the range [0, 1], where a value between 0 and 1 means that the item belongs

to the fuzzy set only partially (see Section 3.1 for a more extensive introduction). For a fuzzy predicate `young(age)`, the 26-year old person would be selected with a membership degree a little bit less than 1. Furthermore, selected items could be ranked on the basis of the extent they satisfy the selection condition. For these reasons, many extensions of SQL that were proposed in the literature preserve the underlying classical relational data model but provide capabilities of soft querying table rows through fuzzy-set theory. Among them, we mention *SQLf* (see [11,12]) and the attempt to implement it (in [25]). We also mention *FQUERY for Access* (see [28,48]), working within Microsoft Access.

The proposal named *SoftSQL* (see [8–10]) also covers the definition of “linguistic predicates”, which can be defined by users through a dedicated statement; then, soft selection conditions can be expressed by means of these user-defined linguistic predicates in the *SELECT* statement; selected rows can be ranked on the basis of their membership degree, obtained by evaluating the soft condition.

In the case the constraint of preserving existing relational databases does not hold, the data model can be extended towards “fuzzy-relational databases” (see [13,33]) for representing uncertain and imprecise data directly within the database, by means of “fuzzy values”. The most remarkable proposal (in our opinion) is *FSQL*, presented in [26,23].

These proposals are compared in several works, such as in [29,30] and in [43]. However, research efforts towards soft querying on relational databases and fuzzy-relational databases ended with the handbook [24], in which all relevant proposals are summarized.

Novel *JSON* document stores (NoSQL databases specifically designed to store *JSON* documents) could provide again fertile ground for research on soft querying databases. Currently, in the literature there are few works that propose fuzzy extensions to query languages on *JSON* document stores. We present them.

The work [1] proposed *fMQL*, an extension of *MQL* (the *MongoDB* query language). The extension is based on *JSON* documents previously labeled with “fuzzy labels”: a fuzzy label is equivalent to a linguistic predicate, because it has an associated membership degree; such labels can be referred in selection conditions. Unfortunately, only one single fuzzy label can be associated to each single *JSON* document; furthermore, the paper did not address how to define them.

The work [34] proposed a technique to translate *fMQL* queries into *fXML*, a fuzzy language for querying XML documents. The positive aspect of this paper is that *fMQL* is more extensively presented than in [1]; the key negative point is that queries are not executed on *JSON* documents but on their XML representations; furthermore, the paper does not address how to deal with membership degrees.

Finally, the work [32] recently proposed an extension of the *MongoDB* data model to support fuzzy values at the level of single document field. Definitely, the authors propose a fuzzy *JSON* document store, which they were able to implement on top of *MongoDB*, by exploiting the internal support to execute JavaScript functions. In our opinion, the technical solution is unusual for the typical approach followed in the database field; nevertheless, it is a first attempt towards fuzzy *JSON* document stores.

The *J-CO* Framework originated from the past work presented in the first part of this section and, currently, is one of the few proposals for performing soft querying on *JSON* data sets. The few proposals presented in the second part of this section show that this research line is actually starting; however, since we are in its early stage, it is necessary to recall past works on relational databases to understand their origins.

3. Background

This section introduces the background for the paper, i.e., basic notions on fuzzy sets and the J-CO Framework.

3.1. Basic notions on fuzzy sets

Soft querying relies on “Fuzzy-Set Theory”, which was introduced by Zadeh (see [46,47]). To let readers that are not familiar with fuzzy sets understand the rest of the paper, this section reports some basic notions.

First of all, consider a non-empty Universe U , which can be either finite or infinite.

Definition 1. A fuzzy set $A \subseteq U$ is a mapping $A : U \rightarrow [0, 1]$ for every item $x \in U$. The value $A(x)$ is referred to as the “membership degree” of x to the A fuzzy set, in that it represents the extent with which x belongs to A .

$A(x) = 1$ means that x fully belongs to A and $A(x) = 0$ means that x does not belong at all to A , while $0 < A(x) < 1$ means that x belongs to A in a partial way (for example, in the case $A(x) = 0.9$, this means that x does not fully belong to A).

Definition 2. Given the universe U and two fuzzy sets A and B , an item $x \in U$ belongs to $I = A \cap B$ with membership degree $I(x) = \min(A(x), B(x))$; an item x belongs to $S = A \cup B$ with membership degree $S(x) = \max(A(x), B(x))$; an item x belongs to $C_A = U - A$ (respectively, to $C_B = U - B$) with membership degree $C_A(x) = 1 - A(x)$ (respectively, $C_B(x) = 1 - B(x)$).

In practice, through the notion of “membership degree”, it is possible to represent fuzzy situations, i.e., situations in which an item $x \in U$ partially belongs to the A set. Why partially? This depends on what the A fuzzy set represents. For example, if x is a person, and A is the set of “Popular People”, we could consider different degrees of popularity: if $A(x) = 1$, this means that the x person is very popular; if $A(x) = 0.5$, we can say that the x person is partially popular; if $A(x) = 0$, the x person is a common person without any degree of popularity.

An obvious question arises: how to evaluate the membership degree? This depends on what items in U represent and on the meaning of the A fuzzy set. In many contexts, it is possible to define a “membership function” $f(x) \in [0, 1]$ that, on the basis of specific properties of x , is able to associate $x \in U$ with a real number in the interval $[0, 1]$, i.e., $A(x) = f(x)$. For example, consider again the universe U of people and consider two people $x_1, x_2 \in U$. Consider now the *Young_People* fuzzy set, which collects people considered as “young”: the fact that $Young_People(x_1) = 0.9$ and $Young_People(x_2) = 0.3$ denotes that neither x_1 nor x_2 are completely young, but x_1 is younger than x_2 . The membership function could be defined as follows.

$$\begin{aligned} Young_People(x) &= 1 && \text{if } x.age \leq 25 \\ Young_People(x) &= (35 - x.age)/10 && \text{if } 25 < x.age < 35 \\ Young_People(x) &= 0 && \text{if } x.age \geq 35 \end{aligned}$$

In words, a person is fully young if the age is up to 25; a person is not young at all if the age is greater than or equal to 35; between 25 and 35, a decreasing linear function from (25, 1) to (35, 0) provides the membership degree. This way, the reader can see that fuzzy sets are able to express vague or imprecise concepts on real-world entities; furthermore, vague or imprecise relationships among real-world entities can be expressed by means of fuzzy sets too.

In this paper, U is the universe set of *JSON* documents. Thus, given a document $d \in U$, the membership degree $A(d)$ denotes the extent with which d belongs to the A fuzzy set. Notice that a

document d can represent a real-world entity; in this case, the name of a fuzzy set represents, in a linguistic way, a category of real-world entities, even those that belong to this category only partially. Similarly, a *JSON* document d can represent an imprecise relationship between real-world entities.

Based on these considerations, we can say that a “soft condition” on *JSON* documents can be expressed by means of “linguistic predicates”, which correspond to fuzzy sets. For example, the search for documents that represent “young and popular people”, with a minimum extent of 0.8, can be performed, provided that the membership degrees of each d document to the fuzzy sets named “Young_People” and “Popular_People” are computed and the fuzzy logical operators AND, OR and NOT are mapped to intersection, union and complement on fuzzy sets (see Definition 2).

3.2. The J-CO framework

The J-CO Framework is a suite of software tools aimed to provide analysts with a powerful instrument to retrieve, transform and integrate collections of *JSON* documents. Fig. 1 depicts the J-CO Framework in relation with the external services that can provide collections of *JSON* documents and storage capabilities. Hereafter, we present the framework in details.

- *J-CO-QL⁺ Engine*. This component actually executes scripts written in the *J-CO-QL⁺* language.
- *J-CO-UI*. Through this tool, users can write *J-CO-QL⁺* scripts, execute them and inspect results.
- *J-CO-DS*. This component (see [39]) manages *JSON* databases able to store huge single documents, such as *GeoJSON* documents (described in [17]). No query language is provided by *J-CO-DS*, because the computational capability is provided by the *J-CO-QL⁺ Engine*.

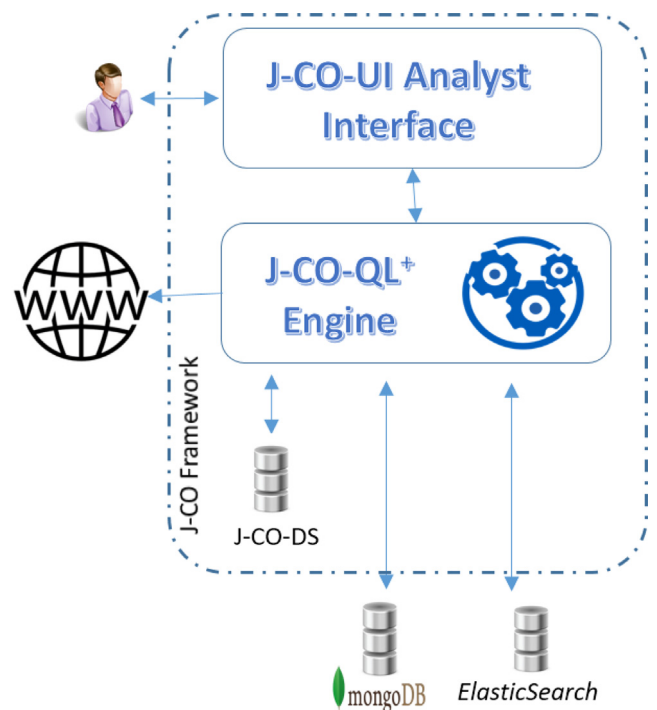


Fig. 1. The J-CO Framework in relation with external services.

The *J-CO-QL⁺ Engine* has been designed to be independent of any *JSON* storage system: in fact, this way *J-CO-QL⁺* can be designed as a powerful query language, which can provide constructs whose execution is independent of the actual computational capabilities provided (if they are provided) by the data sources. The *J-CO-QL⁺ Engine* must interact with external systems only to get data sets and possibly store new data sets.

J-CO-QL⁺ is designed to allow analysts and data engineers to specify complex transformations processes by means of high-level and declarative instructions, which take collections of *JSON* documents as input and possibly generate novel collections of *JSON* documents as output; some of them will be presented in Section 5. Hereafter, we present the data model and the execution model.

J-CO-QL⁺ Data Model. A “collection” is a multi-set (set with multiple occurrences of the same item) of *JSON* documents (see [14]); thus, a *JSON* document is the basic unit to represent and process.

J-CO-QL⁺ Execution Model. *J-CO-QL⁺* relies on a clear and intuitive execution model, hereafter presented.

The query process continuously changes the “query-process state” $s = \langle tc, IR, DBS, FO, UDF \rangle$. The *tc* member is the *temporary collection*, which is a collection of *JSON* documents. *IR* is the “database of *Intermediate Results*”, which is a local storage (that is accessible only by the specific query process) where it is possible to temporarily store collections generated by instructions. *DBS* is the set of descriptors of “database connections”, which specify the databases to connect with. *FO* is the set of “Fuzzy Operators”: defined through a specific statement, they enable the evaluation of membership degrees of documents to fuzzy sets. Finally, *UDF* is the set of user-defined functions that are defined by users to empower instructions.

A query $q = (i_1, \dots, i_n)$ is a sequence of instructions i_j (with $1 \leq j \leq n$). The initial state is $s_0 = \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$ (all its members are empty). Each instruction $i_j \in q$ takes the previous $s_{(j-1)}$ query-process state and generates a new state s_j , possibly changing one

Listing 1: *J-CO-QL⁺* script: JavaScript function named *GeoKmDistance*.

```
1. CREATE JAVASCRIPT FUNCTION GeoKmDistance
  PARAMETERS
    lat1 TYPE Float, lon1 TYPE Float,
    lat2 TYPE Float, lon2 TYPE Float
  PRECONDITION
    lat1 >= -90 AND lat1 <= +90 AND
    lat2 >= -90 AND lat2 <= +90 AND
    lon1 >= -180 AND lon1 <= +180 AND
    lon2 >= -180 AND lon2 <= +180
  BODY {
    var R = 6371; // Earth radius in km
    var dLat = (lat2-lat1) * (Math.PI/180);
    var dLon = (lon2-lon1) * (Math.PI/180);
    var a = Math.sin(dLat/2) * Math.sin(dLat/2) +
            Math.cos(lat1 * (Math.PI/180)) *
            Math.cos(lat2 * (Math.PI/180)) *
            Math.sin(dLon/2) * Math.sin(dLon/2) ;
    var c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
    var d = R * c; // Distance in km
    return d;
  } END BODY;
```

The *J-CO-QL⁺* data model gives a special role to fields in *JSON* documents whose name begins with the $\sim$ character; such field names fully comply with *JSON* naming rules, but they are managed in a special way by the *J-CO-QL⁺ Engine*. Currently, the *J-CO-QL⁺* data model comprises two special fields: $\sim$ fuzzysets and $\sim$ geometry.

The $\sim$ fuzzysets field (if present) represents membership degrees of the *JSON* document to fuzzy sets. It is a flat nested document that behaves as a “key/value” map: the inner-field name is the name of a fuzzy set, while the value is a real number in the range $[0, 1]$, which represents the membership degree of the document to the specified fuzzy set. So, it is possible to manage the membership degree to many fuzzy sets of the same document at the same time.

The $\sim$ geometry field (if present) describes the spatial geometry of the real-world entity described by the document (also said “geo-tagging”); *J-CO-QL⁺* relies on the *GeometryCollection* structure defined by the *GeoJSON* format (see [17]). In this paper, we do not consider geo-tagging; the interested reader can refer to [4,40].

member. In particular, each instruction can use the temporary collection $s_{(j-1)}.tc$ and generate a new temporary collection $s_j.tc$. This way, instructions in the query are written in the same order they are thought by users, by applying the concept of “pipe”.

4. User-defined functions

J-CO-QL⁺ provides users with the capability to write procedural functions to be later called within *J-CO-QL⁺* scripts. This way, the user can implement specific computation methods that are not provided by *J-CO-QL⁺* as built-in functions. This functionality greatly improves flexibility of *J-CO-QL⁺* and can be exploited (as we will show in Section 5) to foster soft querying.

Differently from our previous work (see [19]), now *J-CO-QL⁺* allows for writing user-defined functions in two programming languages, i.e., JavaScript and Java. In the remainder of this section, we show how to define two user-defined functions, named *GeoKmDistance* (for computing the geodesic distance between two points on the Earth globe) and *JaroWinklerSimilarity* (to compute the Jaro-Winkler similarity metric between two strings).

Listing 2: J-CO-QL⁺ script: JavaScript function named JaroWinklerSimilarity.

```

2. CREATE JAVASCRIPT FUNCTION JaroWinklerSimilarity
  PARAMETERS
    s1 TYPE String,
    s2 TYPE String
  BODY {
    s1 = s1.toLowerCase();      s2 = s2.toLowerCase();
    if (s1 == s2)
      return 1.0;

    var len1 = s1.length;      var len2 = s2.length;
    if (len1 == 0 || len2 == 0)
      return 0.0;

    var match = 0;
    var maxDist = Math.floor(Math.max(len1, len2) / 2) - 1;

    var hash_s1 = new Array(s1.length);
    for (var i=0; i<len1; i++)
      hash_s1[i]=0;
    var hash_s2 = new Array(s2.length);
    for (var i=0; i<len2; i++)
      hash_s2[i]=0;

    for (var i = 0; i < len1; i++)
      for (var j = Math.max(0, i-maxDist); j < Math.min(len2, i+maxDist+1); j++)
        if (s1[i] == s2[j] && hash_s2[j] == 0) {
          hash_s1[i] = 1;
          hash_s2[j] = 1;
          match++;
          break;
        }

    if (match == 0)
      return 0.0;

    var t = 0;                  var point = 0;

    for (var i = 0; i < len1; i++)
      if (hash_s1[i] == 1) {
        while (hash_s2[point] == 0)
          point++;
        if (s1[i] != s2[point++])
          t++;
      }

    t /= 2;
    var jaroDistance = ( match/len1 + match/len2 + (match - t)/match ) / 3.0;
    if (jaroDistance > 0.7) {      // Winkler adjustment
      var prefix = 0;
      for (var i = 0; i < Math.min(s1.length(), s2.length()); i++) {
        if (s1.charAt(i) == s2.charAt(i))
          prefix++;
        else
          break;
      }

      prefix = Math.min(4, prefix);
      jaroDistance += 0.1 * prefix * (1 - jaroDistance);
    }
    return jaroDistance;
  } END BODY;

```

4.1. JavaScript functions

First of all, we show how to define JavaScript functions in *J-CO-QL*⁺ through the `CREATE JAVASCRIPT FUNCTION` statement. Listing 1 reports the JavaScript definition of the `GeoKmDistance` function, which computes the geodesic distance between two points, on the basis of their latitude and longitude.

- The `PARAMETERS` clause declares the formal parameters. In this case, `lat1` and `lon1` represent, respectively, the latitude and longitude of the first point, while `lat2` and `lon2` represent, respectively, the latitude and longitude of the second point. All parameters are floating-point numbers.
- The `PRECONDITION` clause specifies a condition that must be met by actual parameters to execute the function: only if the precondition is true, then the function is executed, otherwise an error is raised and the function is not executed. In this case, latitudes must be within the $[-90^\circ, +90^\circ]$ range, while longitudes must be within the $[-180^\circ, +180^\circ]$ range.
- The `BODY` section contains the actual JavaScript code, which in our case calculates the geodesic distance (in *km*) from the coordinates received as actual values for the parameters defined in the `PARAMETERS` clause, by applying the *Haversine formula*³.

- The optional `PRECONDITION` clause is not present: if missing, the function body is always executed.
- In the `BODY` section, notice that the code is not trivial at all and the computation is heavily procedural. The function returns a real number in the $[0, 1]$ range that denotes the similarity degree of the compared strings: 1 denotes that the two strings are equal; 0 denotes that the two strings are completely different. For the sake of space, we do not explain the code.

4.2. Java functions

J-CO-QL⁺ now provides the `CREATE JAVA FUNCTION` statement for defining Java functions. Java programmers could object that simple functions cannot be defined in Java; indeed, this is true, because the corresponding construct is a class method. Nevertheless, from the point of view of *J-CO-QL*⁺, we are introducing a statement for defining functions to invoke within *J-CO-QL*⁺ expressions; this is why we talk about “Java functions”.

Listing 3 reports the Java version of the `GeoKmDistance` function.

- The `PARAMETERS` and `PRECONDITION` clauses are identical to those for defining JavaScript functions and behave in the same way.

Listing 3: *J-CO-QL*⁺ script: Java function named `GeoKmDistance`.

```
1. CREATE JAVA FUNCTION GeoKmDistance
  PARAMETERS
    lat1 TYPE double, lon1 TYPE double,
    lat2 TYPE double, lon2 TYPE double
  PRECONDITION
    lat1 >= -90 AND lat1 <= +90 AND
    lat2 >= -90 AND lat2 <= +90 AND
    lon1 >= -180 AND lon1 <= +180 AND
    lon2 >= -180 AND lon2 <= +180
  CLASS MyGeoClass
  CLASS BODY {
    public static double GeoKmDistance ( double lat1, double lon1,
                                         double lat2, double lon2 ) {
      int R = 6371;
      double dLat = (lat2-lat1) * (Math.PI/180);
      double dLon = (lon2-lon1) * (Math.PI/180);
      double a = Math.pow(Math.sin(dLat/2), 2) +
        Math.cos(lat1 * (Math.PI/180)) *
        Math.cos(lat2 * (Math.PI/180)) *
        Math.pow(Math.sin(dLon/2), 2);
      double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
      double d = R * c;
      return d;
    }
  } END BODY;
```

More complex functions can be defined. This is the case of the `JaroWinklerSimilarity` function, defined in Listing 2. This function evaluates the similarity degree of two strings on the basis of the *Jaro-Winkler* metric ([27,44,45]).

- In Listing 2, the `PARAMETERS` clause specifies two formal parameters: they are the two strings (named `s1` and `s2`) to be compared.

- The user-defined function is mapped to a method in a Java class. Thus, the `CLASS` clause specifies the name of the Java class.
- The `CLASS BODY` clause reports the body of the Java class under definition. Here, as many methods as necessary can be defined, with one single constraint: there must be a static and public method with the same name of the user-defined function under definition and with compatible parameters.

Notice that in Listing 3 we used the same line number (the 1 on the left-hand side) as in Listing 1, to stress that the two function definitions are interchangeable: it is responsibility of the *J-*

³ *Haversine formula* to calculate distance between two geo-referenced points, accessed on 16/02/2023: https://en.wikipedia.org/wiki/Haversine_formula

CO-QL⁺ Engine to properly execute them. In the rest of the script, the user is totally unaware of the specific implementation language.

JaroWinklerSimilarity public method, which the function is mapped to.

Listing 4: J-CO-QL⁺ script: Java function named JaroWinklerSimilarity.

```
2. CREATE JAVA FUNCTION JaroWinklerSimilarity
PARAMETERS
  s1 TYPE String,      s2 TYPE String
CLASS MyStringClass
CLASS BODY {
  public static double JaroWinklerSimilarity(String s1, String s2){
    if ((s1 == null) || (s2 == null))
      return 0;
    s1 = s1.toLowerCase();          s2 = s2.toLowerCase();
    double jaro_dist = jaroDistance(s1, s2);
    if (jaro_dist > 0.7) {
      int prefix = 0;
      for (int i = 0; i < Math.min(s1.length(), s2.length()); i++) {
        if (s1.charAt(i) == s2.charAt(i))
          prefix++;
        else
          break;
      }
      prefix = Math.min(4, prefix);
      jaro_dist += 0.1 * prefix * (1 - jaro_dist);
    }
    return jaro_dist;
  }

  private static double jaroDistance(String s1, String s2)
  {
    if (s1 == s2)
      return 1.0;
    int len1 = s1.length();          int len2 = s2.length();
    if (len1 == 0 || len2 == 0)
      return 0.0;
    int max_dist = (int)Math.floor(Math.max(len1, len2) / 2) - 1;
    int match = 0;
    int hash_s1[] = new int [s1.length()];  int hash_s2[] = new int[s2.length()];
    for (int i = 0; i < len1; i++)
      for (int j = Math.max(0, i - max_dist);
           j < Math.min(len2, i + max_dist + 1); j++)
        if (s1.charAt(i) == s2.charAt(j) && hash_s2[j] == 0) {
          hash_s1[i] = 1;          hash_s2[j] = 1;
          match++;                  break;
        }
    if (match == 0)
      return 0.0;
    double t = 0;          int point = 0;
    for (int i = 0; i < len1; i++)
      if (hash_s1[i] == 1) {
        while (hash_s2[point] == 0)
          point++;
        if (s1.charAt(i) != s2.charAt(point++))
          t++;
      }
    t /= 2;
    return ( ((double)match) / ((double)len1)
            + ((double)match) / ((double)len2)
            + ((double)match - t) / ((double)match) ) / 3.0;
  }
} END BODY;
```

Listing 4 reports the Java version of the JaroWinklerSimilarity function. Notice that a private method (named jaroDistance) is defined within the class: this method is invoked by the

4.3. Semantic model

In Section 3.2, we introduced the execution model and the concept of “query-process state” *s*. Within *s*, the UDF member contains

the descriptors of the user-defined functions. Hereafter, we present the related semantic model.

- A “JavaScript-function descriptor” *jsfd* describes a JavaScript function. It is a tuple $jsfd = \langle name, params, precondition, body \rangle$, where *name* is the function name, *params* is the list of the formal parameters, *precond* is the precondition and *body* is the function body.
- A “Java-function descriptor” *jfd* describes a Java function. It is a tuple $jfd = \langle name, params, precondition, class_name, class_body \rangle$, where *name* is the function name, *params* is the list of the formal parameters, *precond* is the precondition, *class_name* is the name of the Java class and *class_body* is the body of the class.

The *s.UDF* member in the *s* query-process state contains both types of descriptors. Indeed, the scope of the user-defined functions is unique and the *name* member of descriptors uniquely identifies the function, independently of their implementation language. The semantic model is the following one.

- When a new user-defined function is defined, its descriptor *ufd* (either a *jsfd* or a *jfd*) is created.
- If *s.UDF* does not contain a descriptor *d* such that $d.name = ufd.name$, then *ufd* is added to *s.UDF* and the function becomes available for later use.
- If *s.UDF* already contains a descriptor *d* such that $d.name = ufd.name$, then *d* is removed from *s.UDF* and *ufd* is added to *s.UDF* in place of *d*; indeed, user-defined functions can be redefined, independently of the implementation language.

5. Case study and J-CO-QL Plus script

We now show how *J-CO-QL** can integrate user-defined functions for soft querying *JSON* data sets.

We defined a case study that uses one data sets concerning public places in the city of Manchester (UK), gathered for experiments conducted in [42] and in [21]: the first one was provided by *Google Places* (a side service of *Goggle Maps*); the second one was provided by *Facebook*, which was queried to provide pages describing public places. The problem we address in our case study is a problem of data integration.

Problem 1. Consider a set *G* of *JSON* documents that represent descriptors of public places provided by *Google Places*; similarly, consider a set *F* of *JSON* documents that are descriptors of pages created by owners/managers of public places on *Facebook*. The problem to address is “finding the pairs of descriptors (g_i, f_j) (with $g_i \in G$ and $f_j \in F$), such that the two places are close (on the basis of their geodesic distance) and their names are similar (on the basis of their string similarity)”.

To address [Problem 1](#), we wrote a *J-CO-QL** script that is divided in two parts.

- **Part 1**, reported in Listing 5 and explained in Section 5.1, defines two fuzzy operators, which will be exploited in part 2 to evaluate membership degrees of *JSON* documents to fuzzy sets.
- **Part 2**, reported in Listing 6 and explained in Section 5.2, (i) retrieves data from external *JSON* stores, (ii) integrates them in one single collection, (iii) performs a soft query on aggregated documents and (iv) saves the resulting collection into a *JSON* store.

Listing 5: J-CO-QL⁺ Script: Defining Fuzzy Operators.

```

3. CREATE FUZZY OPERATOR Close
  PARAMETERS
    lat1 TYPE Float, lon1 TYPE Float,
    lat2 TYPE Float, lon2 TYPE Float
  PRECONDITION
    lat1 >= -90 AND lat1 <= +90 AND
    lat2 >= -90 AND lat2 <= +90 AND
    lon1 >= -180 AND lon1 <= +180 AND
    lon2 >= -180 AND lon2 <= +180
  EVALUATE GeoKmDistance( lat1, lon1,
                          lat2, lon2 )

  POLYLINE
    [ (0.00, 1.00), (0.03, 1.00),
      (0.10, 0.80), (0.40, 0.30),
      (0.80, 0.05), (1.00, 0.00) ];

4. CREATE FUZZY OPERATOR Similar
  PARAMETERS
    st1 TYPE String,
    st2 TYPE String
  EVALUATE JaroWinklerSimilarity( st1, st2 )
  POLYLINE
    [ (0.00, 0.00), (0.29, 0.00),
      (0.30, 0.30), (1.00, 1.00) ];

```

5.1. Fuzzy operators

*J-CO-QL** provides soft-querying capabilities through the notion of “fuzzy operator”: a fuzzy operator evaluates a membership degree from the actual values of parameters (that could be values of document fields). Fuzzy operators are defined by means of the *CREATE FUZZY OPERATOR* statement, before their use.

Listing 5 reports the *J-CO-QL** instructions that define two fuzzy operators. Hereafter, we describe them.

The fuzzy operator defined on line 3 of Listing 5 is named *Close*; it evaluates the degree of closeness of two geographical points. It is defined as follows.

- The *PARAMETERS* clause declares the formal parameters. In this case, *lat1* and *lon1* represent, respectively, the latitude and longitude of the first point, while *lat2* and *lon2* represent, respectively, the latitude and longitude of the second point. All parameters are floating-point numbers.
- The *PRECONDITION* clause behaves as in user-defined functions. In this case, it checks whether coordinates are valid. In case it is not met, an error is raised and the operator is not evaluated.
- The *EVALUATE* clause defines the mathematical expression whose resulting value is used to obtain the actual membership degree. Specifically, the expression calls the *GeoKmDistance* user-defined function defined on line 1 (Listing 1 or Listing 3) to obtain the distance (in *km*) between two different points.
- The *POLYLINE* clause defines the *membership function* of the fuzzy operator as a polyline. It is described by a list of $n + 1$ points (x_i, y_i) , with $0 \leq i \leq n$, $x_i < x_{i+1}$ and $0 \leq y_i \leq 1$. When $x < x_0$, the corresponding *y*-value is assumed to be equal to y_0 ; when $x > x_n$, the corresponding *y*-value is assumed to be equal to y_n . [Fig. 2](#) depicts the defined polyline. This approach is very flexible, because it is the value returned by the expression in the *EVALUATE* clause of the fuzzy operator that provides the *x* coordinate of the membership function; the corresponding *y* value is the membership degree provided by the operator.

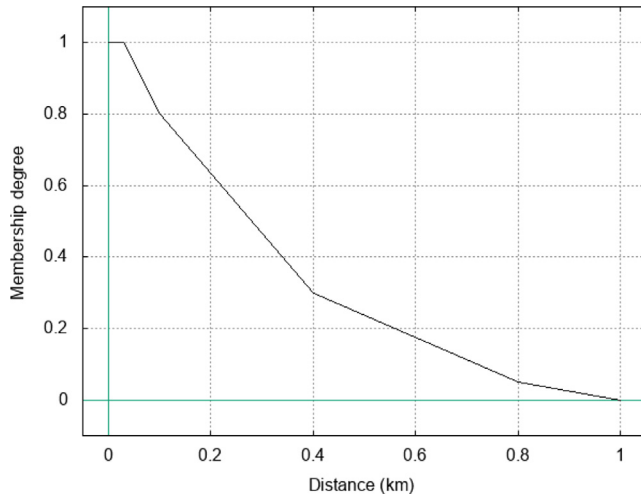


Fig. 2. Membership function of the `Close` fuzzy operator.

Fig. 2 depicts the membership function we defined for the `Close` fuzzy operator. Distances that are less than 1 km are possibly close. Specifically, the membership degree is 1 for distances that are less than 30 m; then, the membership degree decreases with a high slope before 100 m: the slope progressively reduces, until it becomes 0 at 1 km. Notice that the distance is computed by the `EVALUATE` clause, by calling the `GeoKmDistance` user-defined function defined in Listing 1 or Listing 3. The reader can notice how the combined exploitation of user-defined functions and polyline membership functions allows for defining fuzzy operators in a very flexible way.

The second fuzzy operator is defined on line 4 in Listing 5. It is named `Similar` and evaluates the degree of similarity between two strings. We present it in details.

- The operator receives two string parameters, named `str1` and `str2`.
- The `PRECONDITION` clause is not specified; in fact, any string can be received for comparison.
- The `EVALUATE` clause calls the user-defined function named `JaroWinklerSimilarity`, defined in Listing 2 or in Listing 4. The function returns a similarity degree in the $[0, 1]$ range.
- Since the similarity degree returned by the `JaroWinklerSimilarity` function can be already used as the membership degree, a straight segment from (0,0) to (1,1) could be fine, as membership function. However, since we consider a similarity degree less than 0.3 as too low, the actual shape of the polyline (depicted in Fig. 3) is such that the membership degree is 0 from 0 to 0.29; it raises to 0.3 and then it is a linear function with 1 as derivative up to (1,1). This way, similarity degrees between 0.3 and 1.0 becomes the membership degree, while similarity degrees less than 0.29 correspond to the 0 membership degree.

Notice how the semantics of a fuzzy operator is quite rich by itself. However, the possibility to call user-defined functions in the `EVALUATE` expression makes it possible to define fuzzy operators that rely on very complex semantics, where the user-defined function performs complex procedural computations on parameters.

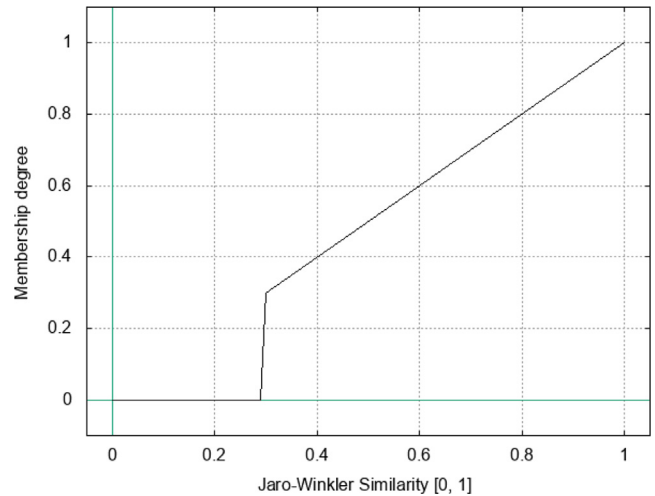


Fig. 3. Membership function of the `Similar` fuzzy operator.

Listing 6: J-CO-QL⁺ Script: Processing data.

```

5. USE DB NeuroComputingDb
   ON SERVER MongoDB 'http://127.0.0.1:27017';

6. JOIN OF COLLECTIONS
   GoogleDescriptors@NeuroComputingDb AS g,
   FacebookDescriptors@NeuroComputingDb AS f
CASE WHERE
  FIELD .g.gLatitude IS NOT NULL AND
  FIELD .g.gLongitude IS NOT NULL AND
  FIELD .f.fbLatitude IS NOT NULL AND
  FIELD .f.fbLongitude IS NOT NULL AND
  GeoKmDistance
    ( .g.gLatitude, .g.gLongitude,
      .f.fbLatitude, .f.fbLongitude ) < 1;

7. FILTER
CASE WHERE
  WITH .g.gName, .f.fbName
GENERATE
CHECK FOR
  FUZZY SET ClosePlaces
    USING Close
      ( .g.gLatitude, .g.gLongitude,
        .f.fbLatitude, .f.fbLongitude ),
  FUZZY SET SimilarNames
    USING Similar(.g.gName, .f.fbName),
  FUZZY SET Wanted
    USING ClosePlaces AND SimilarNames
ALPHACUT 0.8 ON Wanted
BUILD{
  .gId      : .g.gId,
  .fbId     : .f.id,
  .name     : .f.fbName,
  .address  : .f.dbStreet,
  .city     : .f.fbCity,
  .country  : .f.fbCountry,
  .latitude : .f.fbLatitude,
  .longitude: .f.fbLongitude,
  .rank     : MEMBERSHIP_OF(Wanted) }
DEFUZZIFY;

8. SAVE AS RankedPlaces@NeuroComputingDb;

```

5.2. Integrating data

The second part of the *J-CO-QL⁺* script (reported in Listing 6) actually integrates and processes the data sets.

Input Collections. The input collections are stored within a *MongoDB* database, managed by a server running on the same computer where the *J-CO-QL⁺ Engine* runs. These two collections are named *GoogleDescriptors* and *FacebookDescriptors*.

Listing 7: Example of document in the collection named *GoogleDescriptors*.

```
{
  "gId"      : "ChIJmXtxn-qxe0gRy8Fc6sDS3wQ",
  "gName"    : "101 Brasserie & Bar",
  "gAddress" : "Townhouse Hotel Manchester...",
  "gCity"    : "Manchester",
  "gLatitude" : 53.4774472,
  "gLongitude" : -2.2397209
}
```

Listing 8: Example of document in collection named *FacebookDescriptors*.

```
{
  "id"      : 888,
  "idLink"  : "pages/101-Brasserie-...",
  "fbName"  : "101 Brasserie and Bar",
  "fbCity"  : "Manchester",
  "fbCountry" : "United Kingdom",
  "fbStreet" : "101 Portland Street",
  "fbZip"   : "",
  "fbLatitude" : 53.47760716,
  "fbLongitude" : -2.239846765
}
```

Listing 7 reports a sample document that is stored within the *GoogleDescriptors* collection. A variety of fields are present. Here, we are interested in the *gLatitude* and *gLongitude* fields, which describe latitude and longitude of the place, respectively, as well as we are interested in the *gName* field, which denotes the name of the place.

Listing 8 reports a sample document in the *FacebookDescriptors* collection. Among all fields, in this paper, we are interested in the *fbLatitude* and *fbLongitude* fields, whose values are latitude and longitude of the place, respectively, as well as we are interested in the *fbName* field.

Connecting to the database. Looking back at Listing 6, the instruction on line 5 actually connects the query process to the database named *NeuroComputingDb*, which is managed by the *MongoDB* server running on the same machine where the *J-CO-QL⁺ Engine* is running (this way, the IP address of the server is 127.0.0.1). The instruction does not change the temporary collection, which is still empty (since none of the instructions so far executed worked on it); in contrast, a new descriptor is added to the *DBS* member in the query-process state.

Listing 9: Example of document in the temporary collection after line 6 in Listing 6

```
{
  "f" : {
    "id"      : 888,
    "idLink"  : "pages/101-Brasserie-and-Bar...",
    "fbName"  : "101 Brasserie and Bar",
    "fbCity"  : "Manchester",
    "fbCountry" : "United Kingdom",
    "fbStreet" : "101 Portland Street",
    "fbZip"   : "",
    "fbLatitude" : 53.47760716,
    "fbLongitude" : -2.239846765
  },
  "g" : {
    "gId"      : "ChIJmXtxn-qxe0gRy8Fc6sDS3wQ",
    "gName"    : "101 Brasserie & Bar",
    "gAddress" : "Townhouse Hotel Manchester, ...",
    "gCity"    : "Manchester",
    "gLatitude" : 53.4774472,
    "gLongitude" : -2.2397209
  }
}
```

Pairing Descriptors. The instruction on line 6 of Listing 6 gets the two input collections from the database, pairs them and generates a new temporary collection: a document in it is a pair of two descriptors such that the geodesic distance of the places is less than 1 *km*.

Specifically, the instruction specifies the names of the two input collections; for practical convenience, the *GoogleDescriptors* collection is aliased as *g*, while the *FacebookDescriptors* collection is aliased as *f*. This way, each document in the *g* collection is paired to a document in the *f* collection; for each pair, a novel document is generated, which has two fields named *g* and *f* (the value of the *g* field is the source document taken from the *g* collection, while the value of the *f* field is the source document coming from the *f* collection).

Such novel documents are processed by the *CASE WHERE* clause, whose goal is to select only those documents describing pairs of places whose distance is less than 1 *km*.

The selection condition relies on the *IS NOT NULL* predicate, whose syntax is *FIELD fieldReference IS NOT NULL*. These predicates are necessary because it can happen that both *Google Places* and *Facebook* provide null values when they do not know the precise value; for example, if the *Facebook* user that created the page did not provide coordinates, the corresponding fields have the null value. The selection condition in the instruction on line 6 is true for those documents having a not null value for latitudes and longitudes in both the descriptors and the geodesic distance (computed by calling the *GeoKmDistance* user-defined function) is less than 1 *km*. The selected documents constitute the output temporary collection of line 6 (notice the path expression used to refer to fields, such as *.f.fbLatitude*, where the initial dot refers to the root level of the document).

Listing 9 shows the document obtained by pairing the *Google* and *Facebook* descriptors reported in Listings 7 and 8; this document is actually in the temporary collection generated by the instruction on line 6 of the script, because the geodesic distance

is less than 1 km. Notice that latitudes and longitudes are similar but not identical: in fact, it is almost impossible that the two services provide identical coordinates, since their provenance is different and depends on the tools used to measure or set them.

Soft Querying. The instruction on line 7 of Listing 6 actually performs the soft query on the documents in the temporary collection produced by the instruction on line 6. We will discuss this instruction in depth in the dedicated Section 5.3. The output temporary collection produced by the instruction contains flat documents that summarize paired descriptors, such that they satisfy the soft query at a minimal extent. Listing 11 reports the document obtained by processing the sample document reported in Listing 9; notice the presence of the `rank` field, whose value denotes the relevance of the document for the soft query.

Saving the output. Finally, the instruction on line 8 of Listing 6 actually saves the temporary collection produced by the instruction on line 7 into the *MongoDB* database named *NeuroComputingDb*. The collection is saved with name *RankedPlaces*.

Listing 10: Example of document after the `CHECK FOR FUZZY SET` branches on line 7 of Listing 6.

```
{
  "f" : {
    "id" : 888,
    "idLink" : "pages/101-Brasserie-and-Bar...",
    "fbName" : "101 Brasserie and Bar",
    "fbCity" : "Manchester",
    "fbCountry" : "United Kingdom",
    "fbStreet" : "101 Portland Street",
    "fbZip" : "",
    "fbLatitude" : 53.47760716,
    "fbLongitude" : -2.239846765
  },
  "g" : {
    "gId" : "ChIJmXtxn-qxe0gRy8Fc6sDS3wQ",
    "gName" : "101 Brasserie & Bar",
    "gAddress" : "Townhouse Hotel Manchester,...",
    "gCity" : "Manchester",
    "gLatitude" : 53.4774472,
    "gLongitude" : -2.2397209
  },
  "~fuzzyset" : {
    "ClosePlaces" : 1.0,
    "SimilarNames" : 0.95816,
    "Wanted" : 0.95816
  }
}
```

5.3. Soft querying

In this section, we focus on the instruction on line 7 of the *J-CO-QL*⁺ script reported in Listing 6.

- The `FILTER` statement directly works on the temporary collection in the previous query-process state and generates a new temporary collection in the output query-process state. It relies on the `CASE` clause, which is shared with the `JOIN` statement. Soft querying is allowed within a `WHERE` branch, as it is possible to see from Listing 6. Indeed, the `CASE` clause can contain multiple `WHERE` branches, so as to deal with heterogeneous data in the collection; however, this is not the case of our case study, so both the instructions on lines 6 and 7 have only one `WHERE` branch.
- The `WHERE` condition in the instruction on line 7 of Listing 6 relies on the `WITH` predicate. This predicate is true for a document if it contains the specified fields; in this case, the wanted

fields are `gName` (in the *Google* descriptor) and `fName` (in the *Facebook* descriptor).

- The `GENERATE` block processes documents that are selected by the `WHERE` selection condition. Specifically, its sub-clauses are devoted to perform several tasks: (i) dealing with fuzzy sets, by computing the membership degrees of the document to fuzzy sets; (ii) selecting documents on the basis of membership degrees (thus performing the “alpha-cut”); (iii) restructuring the output documents (the `BUILD` block); (iv) removing the computed membership degrees to fuzzy sets, so as to “defuzzify” the document (through the `DEFUZZIFY` option).

Evaluation of fuzzy sets. Membership degrees to fuzzy sets are evaluated by the list of `FUZZY SET` branches in the `CHECK FOR` block. Their behavior is the following.

- A `FUZZY SET` branch evaluates the membership degree to one single fuzzy set, whose name is specified before the `USING` keyword.
- The `USING` keyword specifies the “soft condition” to evaluate so as to obtain the membership degree of the document to the checked fuzzy set.
- If the document does not have the special `~fuzzysets` field, this is added, in such a way it contains one field whose name is the name of the fuzzy set, whose value is the computed membership degree; if the `~fuzzysets` field is already present, a new field is added, denoting the membership degree of the document to the checked fuzzy set.

Consequently, a list of `FUZZY SET` branches progressively extends the evaluated document with membership degrees to fuzzy sets.

The soft condition expressed after the `USING` keyword can refer to previously-computed fuzzy sets (thus exploiting the corresponding membership degrees); furthermore, it can exploit fuzzy operators (see Section 5.1) to compute membership degrees on the basis of properties of the evaluated document.

We are now ready to explain the `CHECK FOR` block and its branches defined on line 7 of Listing 6 in details.

- The first `FUZZY SET` branch evaluates the membership degree of the evaluated document to the fuzzy set named `ClosePlaces`; the `USING` soft condition exploits the `Close` fuzzy operator (defined in Listing 5); since this operator is the only member of the soft condition, the membership degree it computes becomes the actual membership degree of the document to the `ClosePlaces` fuzzy set. This way, the “closeness” relation between the places described by the two paired descriptors has been quantified.
- The second `FUZZY SET` branch evaluates the membership degree of the document under evaluation to the fuzzy set named `SimilarNames`. The `USING` soft condition calls the fuzzy operator named `Similar` (defined in Listing 5), which exploits the Jaro-Winkler similarity metric implemented by the `JaroWinklerSimilarity` user-defined function. Again, since the `Similar` fuzzy operator is the only one member of the soft condition, the membership degree it computes becomes the membership degree of the document under evaluation to the `SimilarNames` fuzzy set. Notice that this fuzzy set actually quantifies the relation between the places described by the paired descriptors on the basis of the similarity of their names.
- The third `FUZZY SET` branch evaluates the membership degree of the document under evaluation to the `Wanted` fuzzy set: the membership degree to this fuzzy set actually expresses the extent to which the document actually meets the wishes of the user, which are expressed by the soft condition.

The soft condition looks for documents that were obtained by pairing descriptors such that the represented places are close and have similar names. The condition refers to the previously-checked fuzzy sets named `ClosePlaces` and `SimilarNames`: this way, they play the role of “linguistic predicates”. The fuzzy AND conjunction (presented in Section 3.1) takes the minimum of the two membership degrees. Listing 10 reports the document shown in Listing 9, as it is transformed by the `CHECK FOR` block; specifically, notice the presence of the `~fuzzysets` field.

Listing 11: Example of document in the output collection named `RankedPlaces`, generated by Line 7 of Listing 6.

```
{
  "address"   : "101 Portland Street",
  "city"      : "Manchester",
  "country"   : "United Kingdom",
  "fbId"      : 888,
  "gId"       : "ChIJmXtxn-qxe0gRy8Fc6sDS3wQ",
  "name"      : "101 Brasserie and Bar",
  "latitude"  : 53.47760716,
  "longitude" : -2.239846765,
  "rank"      : 0.95816
}
```

Alpha-cut. The `ALPHACUT` clause specifies a minimum threshold for the `Wanted` fuzzy set. This way, only documents whose membership degree to the `Wanted` fuzzy set is no less than 0.8 are put into the output collection. Notice that documents are not restructured, but simply kept or discarded; the current structure is still the one shown in Listing 10.

In general, the `ALPHACUT` clause can specify several thresholds for various fuzzy sets. If it is not specified, all documents are put into the output temporary collection.

Generating the final documents. The `BUILD` action specifies the final structure of the output documents generated by the `WHERE` branch. Specifically, each document is flattened, keeping both descriptor identifiers and some key properties. Furthermore, the `rank` field is introduced, by means of the `MEMBERSHIP_OF` built-in function: it provides the membership degree to the fuzzy-set whose name is specified as argument (in this case, the `Wanted` fuzzy set); this way, each output document reports the extent to which it meets the soft condition.

The `BUILD` action does not make any change as far as the special `~fuzzysets` field is concerned: if before the `BUILD` action the document has (does not have) the `~fuzzysets` field, the restructured document has (does not have) it. So, since we want to generate documents without membership degrees to fuzzy sets, the `DEFUZZIFY` option is necessary, to tell the `BUILD` action to remove the `~fuzzysets` field. Documents in the output collection have the structure of the document reported in Listing 11, which derives from the document shown in Listings 9 and 10.

6. Implementation and performance

The reader can see that providing users with the possibility to introduce their own user-defined functions is effective: indeed, it greatly improves the flexibility of the language and its capability to be used in practice.

However, when data sets to process are large, execution time becomes a critical factor, which is influenced by the way functionalities are implemented. Here, we present how user-defined functions are executed by the *J-CO-QL⁺ Engine* and we evaluate performance.

6.1. Implementation

JavaScript Function.

The *J-CO-QL⁺ engine* is implemented in Java.

The *J-CO-QL⁺ Engine* exploits the `javax.script` library: its `ScriptEngine` class provides the `nashorn` implementation of the JavaScript language.

JavaScript is an interpreted programming language: as a result, a program is executed from its textual code. Thus, the *J-CO-QL⁺ Engine* behaves as follows: each time a JavaScript function is invoked, while evaluating a *J-CO-QL⁺* expression, the source code of the function, integrated with the actual parameters, is passed to the instance of the `ScriptEngine` class, which interprets and executes the code and returns control to the *J-CO-QL⁺ Engine* after providing the result of the execution.

Java Function. Java is a compiled programming language, whose binary code is named “Byte Code”: this is a binary format that is independent of the specific processor; to be executed, it needs the “Java Virtual Machine” (JVM) for the specific hardware/software platform.

Thus, execution performance depends on the specific JVM. Since a long time, the official JVM provided by Oracle has incorporated a feature named “Just-in-Time (JIT) Compilation”: when the JVM loads the Byte Code of a class, it is translated into the machine language of the running processor. This way, very good execution times can be obtained; thus, Java can actually be used for intense processing activities.

The *J-CO-QL⁺ Engine* processes Java functions as follows: (i) the code of the class, specified within the `CLASS BODY` clause of the `CREATE JAVA FUNCTION` statement, is saved into a temporary file; the class is compiled through the `JavaCompiler` class provided by the `javax.tools` library; (iii) the output Byte Code is uploaded into main memory (and immediately translated into the machine-language version by the JVM); (iv) when the Java function is invoked, the corresponding method is invoked.

6.2. Performance

In order to evaluate the impact of executing either JavaScript user-defined functions or Java user-defined functions in the *J-CO-QL⁺* script, we presented in Section 5, we performed an experimental evaluation. The results are shown in Table 1; we discuss them hereafter.

- In the “Complete Dataset”, the *F* collection (*Facebook* descriptors) contains 5,738 descriptors, while the *G* collection (*Google Places* descriptors) contains 5,214 descriptors. Among the 29,917,932 possible pairs, 658,286 documents were provided by the `JOIN OF COLLECTIONS` instruction; the soft query on line 7 selected 1,710 documents having the value for the `rank` field no less than 0.8, among which 661 documents have the value 1 (they fully meet the soft query). Thus, 1,049 documents are pairs of descriptors that are potentially of interest, even though they do not match perfectly.

Both using JavaScript and Java user-defined functions, the instruction that takes the highest execution time is the `JOIN OF COLLECTIONS`, followed by the `FILTER` instruction (an order of magnitude faster); the other instructions require a negligible time.

Notice the impressive execution time required to execute the `JOIN OF COLLECTIONS` and the `FILTER` instructions with the JavaScript functions (invoked in both the instructions): the script takes more than 10 h to process $5,738 \times 5,214$ pairs of

Table 1
Execution time of the experiments (in milliseconds).

Line	Instruction	Complete Dataset $ F = 5738 \times G = 5214$		Redux Dataset $ F = 2172 \times G = 1025$		Mini Dataset $ F = 30 \times G = 30$	
		JavaScript	Java	JavaScript	Java	JavaScript	Java
1.	CREATE FUNCTION	1	583	2	576	1	594
2.	CREATE FUNCTION	0	134	0	116	0	122
3.	CREATE FUZZY OPERATOR	0	0	0	0	0	0
4.	CREATE FUZZY OPERATOR	0	0	0	0	0	0
5.	USE DB	2	4	4	2	2	3
6.	JOIN OF COLLECTIONS	34,299,345	172,854	2,168,689	18,320	2,928	1,191
7.	FILTER	3,406,152	19,679	335,926	3,874	468	21
8.	SAVE AS	93	108	55	77	35	35
Total Time (ms)		37,705,593	193,362	2,504,676	22,965	3,434	1,966
Equivalent Total Time		10.474 h	3.223 m	41.745 m	22.965 s	3.434 s	1.966 s

descriptors, meaning that interpreting JavaScript functions is excessively slow. In contrast, with the Java functions, the overall script takes only 3 min.

- From the complete data set, we extracted the “Redux Dataset”: F contains 2,172 descriptors, while G contains 1,025 descriptors, such that a non-empty output collection (152 documents) is obtained.

Notice how the JavaScript version now takes an acceptable time, i.e., 41 min (instead of 10 h); nevertheless, the Java version now takes less than 23 s.

- Finally, we extracted an even smaller data set, named “Mini Dataset”; both F and G contain 30 descriptors, with 11 documents in the output collection.

The general behavior is confirmed: even if the JavaScript version takes only 3.4 sec, the Java version is still faster, taking only 1.9 sec.

A note about the time needed to execute the `CREATE JAVA FUNCTION` (lines 1 and 2 of the “Java” columns): clearly, their execution times are independent of the data set (collections are retrieved from the database by line 6); however, line 1 takes always longer than line 2. We explain this behavior with the time needed by the JVM to load the `javax.tools` library (which compiles source Java code); thus, preparing next classes is faster. Nonetheless, we are talking about hundreds of millisecond, which have a limited weight only with the “Mini Dataset”.

Concluding, we can say that JavaScript user-defined functions provide great flexibility, because they are easier to write than Java classes; however, they are not suitable to process large data sets, due to their poor execution times. On the other side, Java user-defined functions are certainly less flexible to write than JavaScript functions, because they are mapped to Java classes whose programming model is more rigid than that of JavaScript functions; however, thanks to the JIT Compiler, their execution is incredibly fast and allows for effectively exploiting user-defined functions in $J\text{-CO-QL}^+$ scripts for soft querying large data sets in short time.

7. Conclusions

In this paper, we presented how the support for user-defined functions provided by $J\text{-CO-QL}^+$ can increase the level of flexibility provided by the $J\text{-CO}$ Framework to analysts and data engineers. User-defined functions can be written either in the JavaScript language or in the Java language. This way, users can extend the computational capabilities provided by $J\text{-CO-QL}^+$ with domain-specific functions, so as to empower traditional crisp querying and soft querying. By means of a case study, we presented a practical application. The $J\text{-CO}$ Framework has become an effective tool for apply-

ing fuzzy sets and soft querying to *JSON* data sets in a practical way (at the best of our knowledge, no other tool has achieved the same result).

Through the case study, we evaluated performance. In particular, we found that JavaScript functions are extremely slow, due to the fact that JavaScript is an interpreted programming language; so the price for its flexibility in writing functions is slowness. On the other side, Java functions are executed very quickly; the price for this is a higher complexity in writing Java classes (with respect to JavaScript functions).

Evaluation of fuzzy sets and the consequent support to soft querying currently is the main development line of $J\text{-CO-QL}^+$: at the end of its development, $J\text{-CO-QL}^+$ is aimed to provide full support to soft querying, by enabling all the statements to deal with fuzzy sets. In fact, statements for grouping and unnesting (not presented in this paper) have not been extended yet.

A key role will be played by the continued optimization of the $J\text{-CO-QL}^+$ Engine, in particular as far as processing Big Data is concerned: we will exploit the experience we made in [35,38], in the context of blind querying of Open Data portals (see [36,37]) and *JSON* document stores (see [31]).

Nevertheless, the $J\text{-CO}$ Framework can be a valuable execution engine for domain-specific languages to query specific *JSON* structures. We demonstrated that in [18], where we proposed a domain-specific language for soft querying features within *GeoJSON* documents, whose translator generates $J\text{-CO-QL}$ scripts that actually perform soft querying on *GeoJSON* documents.

Moreover, we plan to exploit the $J\text{-CO}$ Framework for performing studies on human sciences. In fact, the idea of devising the framework originated while working in the *Urban Nexus* project (see [16]), which was a joint project of geographers and computer scientists devoted to study mobility through Big Data and social media. Since we had to deal with *JSON* data sets describing tweets of moving users (see [7,6]), we conceived the $J\text{-CO}$ Framework.

The $J\text{-CO}$ Framework is available on a public GitHub repository⁴.

Data availability

Data will be made available on request.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

⁴ GitHub repository of the $J\text{-CO}$ Framework, accessed on 16/02/2023: <https://github.com/JcoProjectTeam/JcoProjectPage>

References

- [1] B.K. Abir, G.T. Amel, Towards fuzzy querying of nosql document-oriented databases, DBKDA 2015 (2015) 163.
- [2] J.C. Anderson, J. Lehnardt, N. Slater, CouchDB: the definitive guide: time to relax, O'Reilly Media Inc., 2010.
- [3] Blair, D.C., 1979. Information retrieval, 2nd ed. c.j. van rijnsbergen. london: Butterworths; 1979: 208 pp. price: \$32.50. Journal of the American Society of Information Science 30, 374–375.
- [4] G. Bordogna, S. Capelli, D.E. Ciriello, G. Psaila, A cross-analysis framework for multi-source volunteered, crowdsourced, and authoritative geographic information: The case study of volunteered personal traces analysis against transport network data, Geo-spatial Information Science 21 (2018) 257–271.
- [5] G. Bordogna, S. Capelli, G. Psaila, A big geo data query framework to correlate open data with social network geotagged posts, The Annual International Conference on Geographic Information Science, Springer (2017) 185–203.
- [6] G. Bordogna, A. Cuzzocrea, L. Frigerio, G. Psaila, M. Toccu, An interoperable open data framework for discovering popular tours based on geo-tagged tweets, International Journal on Intelligent Information and Database Systems 10 (2017) 246–268.
- [7] Bordogna, G., Frigerio, L., Cuzzocrea, A., Psaila, G., 2016. An effective and efficient similarity-matrix-based algorithm for clustering big mobile social data, in: 2016 15th IEEE International Conference on Machine Learning and Applications (ICMLA), IEEE. pp. 514–521.
- [8] Bordogna, G., Psaila, G., 2008a. Customizable flexible querying in classical relational databases, in: Handbook of research on fuzzy information processing in databases. IGI Global, pp. 191–217.
- [9] G. Bordogna, G. Psaila, Modeling soft conditions with unequal importance in fuzzy databases based on the vector p-norm, IPMU-2008, Malaga, 2008.
- [10] G. Bordogna, G. Psaila, Soft aggregation in flexible databases querying based on the vector p-norm, Int. J. Uncertainty, Fuzziness Knowl.-Based Syst. 17 (2009) 25–40.
- [11] P. Bosc, O. Pivert, Sqfl: a relational database language for fuzzy querying, IEEE Trans. Fuzzy Syst. 3 (1995) 1–17.
- [12] P. Bosc, O. Pivert, Sqfl query functionality on top of a regular relational database management system, Knowledge Management in Fuzzy Databases. Springer (2000) 171–190.
- [13] P. Bosc, H. Prade, An introduction to the fuzzy set and possibility theory-based treatment of flexible queries and uncertain or imprecise databases, Uncertainty management in information systems. Springer (1997) 285–324.
- [14] Bray, T., 2014. The javascript object notation (json) data interchange format. URL <https://www.rfc-editor.org/rfc/rfc7159.txt>
- [15] P.G. Bringas, I. Pastor, G. Psaila, Can blockchain technology provide information systems with trusted database? the case of hyperledger fabric, in: International Conference on Flexible Query Answering Systems, Springer, Cham, 2019, pp. 265–277.
- [16] Burini, F., Cortesi, N., Gotti, K., Psaila, G., 2018. The urban nexus approach for analyzing mobility in the smart city: Towards the identification of city users networking. Mobile Information Systems 2018.
- [17] Butler, H., Daly, M., Doyle, A., Gillies, S., Hagen, S., Schaub, T., et al., 2016. The geojson format. Internet Engineering Task Force (IETF).
- [18] Fosci, P., Marrara, S., Psaila, G., 2020. Soft querying geojson documents within the j-co framework, in: 16th International Conference on Web Information Systems and Technologies (WEBIST 2020), SCITEPRESS. pp. 253–265.
- [19] P. Fosci, G. Psaila, Powering soft querying in j-co-ql with javascript functions, International Workshop on Soft Computing Models in Industrial and Environmental Applications, Springer (2021) 207–221.
- [20] P. Fosci, G. Psaila, Towards flexible retrieval, integration and analysis of json data sets through fuzzy sets: a case study, Information 12 (2021) 258.
- [21] P. Fosci, G. Psaila, Soft integration of geo-tagged data sets in j-co-ql+, ISPRS International Journal of Geo-Information 11 (2022) 484.
- [22] Fosci, P., Psaila, G., 2022b. Soft spatial querying on json data sets, in: Advances in Databases and Information Systems: 26th European Conference, ADBIS 2022, Turin, Italy, September 5–8, 2022, Proceedings, Springer International Publishing Cham. pp. 223–237.
- [23] J. Galindo, New characteristics in fsq, a fuzzy sql for fuzzy databases, WSEAS Trans. Inform. Sci. Appl. 2 (2005) 161–169.
- [24] J. Galindo, Handbook of research on fuzzy information processing in databases, IGI Global, 2008.
- [25] J. Galindo, J.M. Medina, O. Pons, J.C. Cubero, A server for fuzzy sql queries, International Conference on Flexible Query Answering Systems, Springer (1998) 164–174.
- [26] J. Galindo, A. Urrutia, M. Piattini, Fuzzy databases: Modeling, design, and implementation, IGI Global, 2006.
- [27] M.A. Jaro, Advances in record-linkage methodology as applied to matching the 1985 census of tampa, florida, J. Am. Stat. Assoc. 84 (1989) 414–420.
- [28] Kacprzyk, J., Zadrozny, S., 1995. Fquery for access: Fuzzy querying for a windows-based dbms. Bosc P., Kacprzyk J. (eds), Fuzziness in Database Management Systems. Studies in Fuzziness 5.
- [29] Kacprzyk, J., Zadrozny, S., 2001. Sqfl and fquery for access, in: Proceedings Joint 9th IFSA World Congress and 20th NAFIPS Conference, IEEE. pp. 2464–2469.
- [30] Z.M. Ma, L. Yan, Generalization of strategies for fuzzy query translation in classical relational databases, Inf. Softw. Technol. 49 (2007) 172–180.
- [31] S. Marrara, M. Pelucchi, G. Psaila, Blind queries applied to json document stores, Information 10 (2019) 291.
- [32] J.M. Medina, I.J. Blanco, O. Pons, A fuzzy database engine for mongodb, Int. J. Intell. Syst. Online library (2022).
- [33] J.M. Medina, O. Pons, M.A. Vila, Gefred: A generalized model of fuzzy relational databases, Inf. Sci. 76 (1994) 87–109.
- [34] F. Mehrab, A. Harounabadi, Apply uncertainty in document-oriented database (mongodb) using f-xml, J. Adv. Comput. Res. 9 (2018) 87–101.
- [35] Pelucchi, M., Psaila, G., Toccu, M., The challenge of using map-reduce to query open data, in: 6th DATA COnference; Madrid; Spain; July 24–26, 2017 pages=331–342, year=2017.
- [36] M. Pelucchi, G. Psaila, M. Toccu, Building a query engine for a corpus of open data, WEBIST, in, 2017, pp. 126–136.
- [37] M. Pelucchi, G. Psaila, M. Toccu, Enhanced querying of open data portals, in: International Conference on Web Information Systems and Technologies, Springer, Cham, 2017, pp. 179–201.
- [38] M. Pelucchi, G. Psaila, M. Toccu, Hadoop vs. spark: Impact on performance of the hammer query engine for open data corpora, Algorithms 11 (2018) 209.
- [39] Psaila, G., Fosci, P., 2018. Toward an anyist-oriented polystore framework for processing json geo-data, in: Int. Conf. on Applied Computing 2018, Budapest; Hungary, 21–23 October 2018, IADIS. pp. 213–222.
- [40] G. Psaila, P. Fosci, J-co: A platform-independent framework for managing geo-referenced json data sets, Electronics 10 (2021) 621.
- [41] G. Psaila, S. Marrara, A first step towards a fuzzy framework for analyzing collections of json documents, IADIS AC 2019 (2019) 19–28.
- [42] G. Psaila, M. Toccu, A fuzzy technique for on-line aggregation of pois from social media: Definition and comparison with off-line random-forest classifiers, Information 10 (2019) 388.
- [43] Urrutia, A., Tineo, L., Gonzalez, C., 2008. Fsql and sqfl: Towards a standard in fuzzy databases, in: Handbook of Research on Fuzzy Information Processing in Databases. IGI Global, pp. 270–298.
- [44] Winkler, W.E., 1990. String comparator metrics and enhanced decision rules in the fellegi-sunter model of record linkage, in: Proceedings of the Section on Survey Research Methods, American Statistical Association. p. 354–359.
- [45] Winkler, W.E., 1999. The state of record linkage and current research problems, in: Statistical Research Division, US Census Bureau, Citeseer.
- [46] L.A. Zadeh, Fuzzy sets, Inform. Control 8 (1965) 338–353.
- [47] L.A. Zadeh, The concept of a linguistic variable and its application to approximate reasoning-i, Inform. Sci. 8 (1975) 199–249.
- [48] Zadrozny, S., Kacprzyk, J., 1996. Fquery for access: towards human consistent querying user interface, in: Proceedings of the 1996 ACM symposium on Applied Computing, pp. 532–536.